

Project Proposal:

Enterprise IT Asset & Security Management (EITASM)

Course:

420-921-VA Database

Professor:

Ms. Silviya Lyubomirova Paskaleva

Group Members:

Danny Gilberto

Nafiseh Khosravi Rad

Kwok Yiu Wong

Date:

May 7, 2026

Table of Contents

- 1. Project Objective..... 4
 - 1.1. Core Technical Objectives 4
 - 1.2. Advanced Database Features 4
 - 1.3. Project Methodology..... 5
 - 1.4. The Bottom Line 5
- 2. Expectation Outcomes 6
- 3. Requirements Specifications..... 7
 - 3.1. Functional Requirements 7
 - 3.2. Non-Functional Requirements 7
 - 3.3. Initial Mind Mapping (Data Architecture)..... 8
 - 3.4. UML Use Case Diagram..... 8
- 4. Agile Structure & Project Roadmap 9
- 5. Database Administration Plan.....11
 - 5.1. Backup & Recovery Strategy.....11
 - 5.2. Management of Authorization (Confidentiality)11
 - 5.3. Flow Logic 12
- 6. Technical Stack Justification & Requirements 14
- 7. Entity-Relationship Diagram (ERD) Logic 15
 - 7.1. Core Entities and Attributes: 15

7.2. Relationships:	15
7.3. Normalization Breakdown (3NF)	16
7.4. Database Logic & Constraints	16
7.5. Implementation for our Trello Board	17
7. 6. Data Dictionary & Field Constraints Specifications	17
7.7. Advanced SQL Logic & Automation	18
8. Database Operational Logic	20
9. Implementation Checklist	21
10. Software and Technologies	22
10.1. Database & Technical Implementation	22
10.2. Design & Modeling Tools	22
10.3. Project Management & Methodology	23
10.4. Application Architecture (Optional)	23
10.5. Documentation & Version Control	23
11. Team Roles & Responsibilities	24

1. Project Objective

The objective is to design and implement a centralized Relational Database Management System (RDBMS) to track, manage, and secure IT assets for a mid-to-large enterprise of 500 ~ 700 employees. The system will bridge the gap between HR (who owns the employee data), IT (who owns the hardware/software), and Security (who manages incidents and compliance) departments to automate asset tracking, software licensing, and security compliance within a single, secure environment.

1.1. Core Technical Objectives

- **Asset & License Lifecycle:** Automated tracking of hardware (laptops, servers) and software utilization, ensuring 100% visibility of company property.
- **Automated Security Enforcement:** Implementation of **SQL Triggers** to prevent unauthorized asset assignment (comparing Employee Clearance vs. Asset Security levels) and to flag assets automatically during security incidents.
- **Data Integrity & Normalization:** A robust schema designed to **3rd Normal Form (3NF)** to eliminate redundancy and ensure referential integrity across Personnel, Inventory, and Incident logs.

1.2. Advanced Database Features

- **Confidentiality & RBAC:** Utilization of **SQL Views and Roles** to enforce a "Need to Know" principle, masking sensitive personnel data (like clearance levels or potential salary data) from general IT staff.

- **Business Logic Automation: Stored Procedures** for complex workflows, such as employee onboarding (bulk asset/license provisioning) and offboarding.
- **Disaster Recovery:** A formal Database Administration plan including automated **Backup and Restore** scripts to ensure 99.9% data availability.

1.3. Project Methodology

The project is managed via **Agile Kanban (Trello)** and version-controlled on **GitHub**. Development is divided into three Sprints:

1. **Sprint 1:** Modeling & Design (ERDs, Use Cases, Mind Mapping).
2. **Sprint 2:** Core Construction (Schema DDL, 700+ record Data Seeding).
3. **Sprint 3:** Advanced Logic (Triggers, Procedures, Security Audit Logging).

1.4. The Bottom Line

EITASM moves beyond a simple inventory list by transforming a database into an active security participant that enforces corporate policy at the data layer.

2. Expectation Outcomes

- **Centralized Asset Tracking:** 100% visibility of hardware (laptops, servers) and software licenses.
- **Security Compliance:** Automated triggers to flag unauthorized software or expired security patches.
- **Role-Based Access Control:** Strict data confidentiality utilizing SQL Views and Schemas to ensure departments (HR, IT, Security) only access 'need-to-know' data.
- **Audit Readiness:** Automated SQL scripts to generate compliance reports for stakeholders.

3. Requirements Specifications

3.1. Functional Requirements

- **FR1: Asset Lifecycle Management:** The system shall allow authorized IT staff to create, update, and decommission hardware assets.
- **FR2: Automated Assignment:** The system must provide a mechanism to link an Employee ID to an Asset ID and a Software License ID simultaneously.
- **FR3: Inventory Alerts:** The system shall generate automated alerts when software license "seats" are 90% utilized or within 30 days of expiration.
- **FR4: Incident Reporting:** Users must be able to log security incidents against specific Serial Numbers, which automatically updates the asset status to "Under Investigation."

3.2. Non-Functional Requirements

- **NFR1: Security & Confidentiality:** Access to the Personnel_Salary and Security_Incidents tables must be restricted via SQL Roles and Views.
- **NFR2: Data Integrity:** The system must enforce referential integrity; for example, an asset cannot be assigned to an Employee ID that does not exist in the Personnel table.
- **NFR3: Availability:** The database must support 24/7 read access for automated security monitoring scripts.
- **NFR4: Recoverability:** The system must include a documented recovery script to restore data to a "last known good state" within 30 minutes of a failure.

- **NFR5: Data Atomicity (ACID):** All asset assignments must be handled as a single transaction; if the hardware assignment fails, the software license provisioning must also roll back.

3.3. Initial Mind Mapping (Data Architecture)

To visualize the scope, the database will be branched into four primary clusters:

- **Personnel:** Employee ID, Name, Department, Role, Security Clearance Level.
- **Hardware Assets:** Serial Number, Asset Type (Workstation/Server/Mobile), Purchase Date, Assigned User, Physical Location.
- **Software & Licenses:** Software Name, License Key, Version, Expiry Date, Seat Count.
- **Security & Incidents:** Incident ID, Severity, Linked Asset, Resolution Status, Timestamp.

3.4. UML Use Case Diagram

The following illustrates the interactions between different enterprise roles and the EITASM system:

- **Employee:** Views assigned assets and reports hardware issues.
- **IT Technician:** Manages inventory, assigns hardware, and installs software.
- **Security Admin:** Reviews security logs, manages clearance levels, and audits compliance.
- **System (Automated):** Triggers alerts for license expirations and clearance mismatches.

4. Agile Structure & Project Roadmap

Our project execution follows the Agile Kanban methodology., it will be managed via a **Trello Board** and divided into three distinct **Sprints**.

Sprint 1: Analysis, Modeling, & Design by Completing of Mind Maps, Functional Requirements, and ER Diagrams. (Week 1-2)

- **Task 1:** Finalize Functional Requirements (e.g., "The system must alert when a software license is 30 days from expiry").
- **Task 2:** Create Use Case Diagrams (Defining interactions for Employee, IT Tech, and Security Admin).
- **Task 3:** Design the ER Diagram (Normalizing tables to 3rd Normal Form).
- **Deliverable:** Approved Design Document & GitHub Repo initialization.

Sprint 2: Core Development & Construction by Creating SQL Schema, Seeding Data (700 mock records), and Implementing Primary/Foreign Key. (Week 3-4)

- **Task 1:** Write SQL Scripts for Schema Creation (Tables, Primary Keys, Foreign Keys).
- **Task 2:** Data Seeding (Populating the database with mock data for 700 employees).
- **Task 3:** Develop Core Queries (Search assets by department, list all software assigned to a specific user).
- **Deliverable:** Working Database Instance.

Sprint 3: Advanced Logic, Features, Testing, & Security by implementing Stored Procedures for onboarding, Security Triggers, and Authorization management. (Week 5-6)

- **Task 1:** Implement **Stored Procedures** for complex tasks (e.g., Offboarding an employee and releasing all their licenses).
- **Task 2:** Implement **Triggers** for security (e.g., prevent assigning a 'High Security' laptop to a 'Low Clearance' employee).
- **Task 3:** Management of Authorization (Creating Database Roles and Permissions).
- **Deliverable:** Final Presentation & Documentation.

Final Review: Verification against the "Student Checklist" and final presentation preparation.

5. Database Administration Plan

5.1. Backup & Recovery Strategy

Following the requirements of the 420-231-VA syllabus, we will implement a robust backup strategy:

- **Full Backups:** Executed weekly to capture the entire schema and data state.
- **Differential Backups:** Executed daily to capture changes since the last full backup.
- **Restoration:** A dedicated SQL script will be maintained in our GitHub repository to automate the RESTORE DATABASE command in the event of data corruption.
- **Method:** We will utilize the native SQL `BACKUP DATABASE` command to generate `.bak` files.
- **Version Control:** All DDL (Data Definition Language) scripts and DML (Data Manipulation Language) scripts will be stored on GitHub to allow for a complete schema reconstruction if the physical database instance is lost.
- **Restoration Drill:** As part of our Sprint 3 testing, we will perform a "Clean Slate" test where we drop the database and recreate it entirely using our GitHub script repository.

5.2. Management of Authorization (Confidentiality)

To establish data confidentiality, we will implement Role-Based Access Control (RBAC):

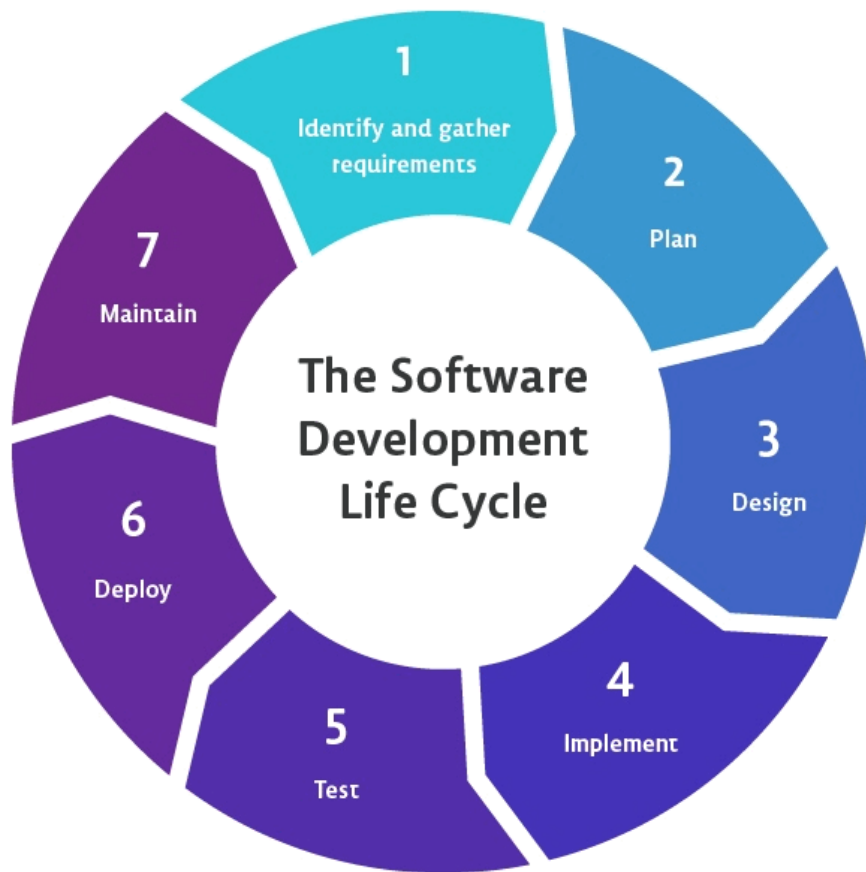
- **Views:** We will use SQL Views to mask sensitive information (e.g., `vw_Public_Personnel` will exclude home addresses and clearance levels).
- **Schemas:** Data will be partitioned into HR, IT, and SEC schemas with restricted cross-schema permissions.

- **The "Need to Know" Principle:** We will implement three specific Database Roles:
 - **IT_Inventory_Manager:** Full CRUD (Create, Read, Update, Delete) permissions for the Assets and Licenses tables.
 - **Security_Auditor:** Read-only access to all tables, but full access to the **SecurityIncidents** table.
 - **General_HR:** Access restricted to the **Employees** table via a **View** that masks sensitive security clearance data.

5.3. Flow Logic

Asset Assignment Flow

1. **Request:** IT Technician initiates an assignment script for an employee.
2. **Validation (Trigger-Based):** The system executes a **BEFORE UPDATE** trigger on the **HardwareAssets** table. It compares the **ClearanceLevel** of the target employee against the **SecurityRequirement** of the asset.
3. **Result:** If the levels do not match, the trigger raises a RAISERROR (or custom SQL error) and automatically rolls back the transaction, ensuring no unauthorized assignments occur.



Source: <https://pvs-studio.com/en/blog/terms/6732/>

6. Technical Stack Justification & Requirements

- **RDBMS (SQL Server/PostgreSQL):** Chosen for its ability to handle complex relational constraints and stored procedures required for an enterprise of 700 employees.
- **SQL (Structured Query Language):** Used for DDL (Data Definition) and DML (Data Manipulation) to ensure 3rd Normal Form compliance.
- **GitHub:** Utilized for version control of all `.sql` scripts, ensuring the team can collaborate on the schema design asynchronously.
- **3-Tier Architecture:** Our design separates the **Data Tier** from the **Logic Tier** and the **Presentation Tier** ensuring modularity and scalability.
 - **Data Tier:** The RDBMS (SQL Server) where the tables and relational logic reside.
 - **Logic Tier** (Stored Procedures/Triggers): Encapsulated in the database itself via Stored Procedures and Triggers, ensuring business rules are enforced even if the UI changes.
 - **Presentation Tier** (mock UI/Reports): Represented by the SQL Views and generated reports designed for different enterprise departments (HR, IT, Security).
- **Documentation:** Functional & Non-Functional requirements document.
 - ER Diagram showing Many-to-Many relationships (e.g., Employees to Software).
 - Mind map of the enterprise ecosystem.
- **Security:** Implementation of VIEWS to hide sensitive employee data from general IT technicians.

7. Entity-Relationship Diagram (ERD) Logic

For an enterprise of 700 employees, the database must be normalized to ensure data integrity and prevent redundancy.

7.1. Core Entities and Attributes:

- **Employees (Eliminates transitive dependencies by moving Dept details to the Departments table):** EmployeeID (PK), FirstName, LastName, DepartmentID (FK), Email (Unique), ClearanceLevel (1-5).
- **Departments:** DepartmentID (PK), DeptName, ManagerID (FK to Employees).
- **HardwareAssets:** AssetID (PK), SerialNumber (Unique), Model, AssetTypeID (FK), PurchaseDate, SecurityRequirement (1-5), CurrentStatus, AssignedToEmployeeID (FK).
- **SoftwareLicenses:** LicenseID (PK), SoftwareName, Version, TotalSeats, ExpiryDate.
- **LicenseAssignments:** AssignmentID (PK), LicenseID (FK), EmployeeID (FK), InstallDate.
- **SecurityIncidents:** IncidentID (PK), AssetID (FK), ReporterID (FK to Employees), SeverityLevel (FK - lookup table for Low/Med/High), Description, DateReported, ResolutionStatus (Open, Investigating, Resolved), CreatedAt.

7.2. Relationships:

- **One-to-Many:** One Department has many Employees.
- **One-to-Many:** One Employee can be assigned multiple Hardware Assets.

- **Many-to-Many:** Employees and Software Licenses (resolved via the `LicenseAssignments` junction table).
- **One-to-Many:** One Hardware Asset can have multiple Security Incidents over its lifecycle.

7.3. Normalization Breakdown (3NF)

- **1st Normal Form (1NF):** Each field contains atomic values (e.g., `FirstName` and `LastName` are separate), and every table has a Primary Key.
- **2nd Normal Form (2NF):** All non-key attributes are fully functional dependent on the Primary Key. By using junction tables like `LicenseAssignments`, we ensure that software details aren't duplicated for every employee.
- **3rd Normal Form (3NF):** We have removed **transitive dependencies**.

Example #1: In the `HardwareAssets` table, we store a `VendorID` rather than the Vendor's name and support contact info. This ensures that if a vendor changes their support number, we update it once in the `Vendors` table, satisfying 3NF by removing transitive dependencies.

Example #2: `SeverityLevel` is moved to a lookup table so that the description of "High Severity" isn't repeated across 500 incident rows.

7.4. Database Logic & Constraints

The flow chart below illustrates the lifecycle of an asset from procurement to decommissioning, including the automated security checks required by our course.

The Operational Flow:

1. **Procurement:** A new laptop is added to `HardwareAssets`.

2. **Onboarding (Agile Sprint 2 Task):** When a new employee is hired, a **Stored Procedure** checks for an "Available" asset.
3. **Security Check (Constraint/Trigger):** Before the assignment is finalized, a **Trigger** verifies the `ClearanceLevel`. If the employee is Level 1 but the asset is marked for Level 3, the transaction fails.
4. **Maintenance/Incident:** If a `SecurityIncident` is logged against an `AssetID`, the `HardwareAssets.CurrentStatus` is automatically updated to "Under Investigation" via a trigger.

7.5. Implementation for our Trello Board

- **Sprint 2 Goal:** Create the SQL schema using `CREATE TABLE` statements with proper `FOREIGN KEY` constraints.
- **Confidentiality Task:** Use **SQL Views** to create a "Public Employee Directory" that excludes `ClearanceLevel` and `Salary` (if added), fulfilling the "Management of Authorization" requirement in our syllabus.

7.6. Data Dictionary & Field Constraints Specifications

- **Integrity Constraints:** All Primary Keys will be set to `IDENTITY(1,1)` or `AUTO_INCREMENT`. Foreign Keys will use `ON DELETE SET NULL` to ensure that deleting an employee does not accidentally purge valuable asset history and to preserve asset history if a personnel record is removed.
- **Employees Table:**
 - `EmployeeID`: INT IDENTITY(1,1) (Primary Key)

- `ClearanceLevel`: TINYINT (Check Constraint: 1-5)
- `Email`: VARCHAR(255) (Unique Constraint)
- **HardwareAssets Table:**
 - `AssetID`: INT IDENTITY(1,1) (Primary Key)
 - `SerialNumber`: VARCHAR(50) (Unique Constraint)
 - `PurchaseDate`: DATE
 - `SecurityRequirement`: TINYINT (Check Constraint: 1-5)
 - `CurrentStatus`: VARCHAR(20) (Default: 'Available')
- **SecurityIncidents Table:**
 - `IncidentDescription`: VARCHAR(MAX)
 - `CreatedAt`: DATETIME (Default: GETDATE())

7.7. Advanced SQL Logic & Automation

- **Stored Procedure:** `sp_AssignAssetToEmployee` This procedure will handle the transaction of checking asset availability, verifying the employee exists, and updating the `HardwareAssets` status in a single atomic operation.
- **Trigger:** `tr_UpdateAssetStatusOnIncident` A trigger on the `SecurityIncidents` table that automatically changes the linked asset's status to "Flagged" or "Maintenance" the moment a high-severity incident is logged.

Script: `Daily_License_Audit` A scheduled script that performs a JOIN between `SoftwareLicenses` and `LicenseAssignments` to identify products where the seat count has been exceeded.

8. Database Operational Logic

This flow chart describes how data moves through the system during a standard "Asset Procurement and Assignment" lifecycle, following the SDLC principles in our lecture notes.

The Workflow:

1. **Entry Point:** New hardware is purchased. Data is inserted into the `HardwareAssets` table with a status of "Available."
2. **Request Trigger:** HR adds a new employee to the `Employees` table.
3. **Asset Allocation:** An IT Technician runs a **Stored Procedure** to assign an available laptop to the new `EmployeeID`.
4. **Validation Check:** The system runs a check: Does the `Employee.ClearanceLevel` meet the `Asset.SecurityRequirement`?
 - *If No:* Transaction is rolled back via trigger logic.
 - *If Yes:* The `HardwareAssets` status updates to "Assigned."
5. **Software Provisioning:** The system checks the `SoftwareLicenses` table for available seats. If `TotalSeats > UsedSeats`, a record is created in `LicenseAssignments`.
6. **Audit Logging:** An **Automatic Trigger** captures the transaction details and writes them to a read-only `AuditLogs` table for security compliance.

9. Implementation Checklist

The following technical components will be included in our SQL scripts:

- **Constraints:** Use `NOT NULL`, `UNIQUE` (for Serial Numbers), and `CHECK` constraints (to ensure `ClearanceLevel` is between 1 and 5).
- **Stored Procedure:** Create one called `sp_OnboardEmployee` that handles the insertion of the employee and the initial assignment of a standard "Tech Kit" (Laptop/Mouse/License).
- **Views:** Create a View called `vw_AssetCompliance` that joins Assets, Employees, and Incidents so managers can see at-risk hardware without seeing private employee data (fulfilling the **Data Confidentiality** requirement).
- **Backup Script:** Include a `.sql` script that performs a full backup of the database.

10. Software and Technologies

10.1. Database & Technical Implementation

- **Relational Database Management System (RDBMS):** We will use an RDBMS to define, manipulate, and query our database.
- **Structured Query Language (SQL):** This will be our primary language for creating tables, writing scripts, and developing **Stored Procedures**.
- **Advanced Database Features:** We are expected to implement **Authorization Management** (data confidentiality) and procedures to **Save and Restore** the database.
- **GitHub:** We will create a repository to host our project code and track our commits.

10.2. Design & Modeling Tools

- **Mind Mapping:** Used in the early stages to brainstorm our database entities and requirements.
- **Unified Modeling Language (UML) Diagrams:**
 - **Use Case Diagrams:** To visualize how different users interact with our system.
 - **Entity-Relationship (ER) Diagrams:** To define our table structures and relationships.
- **Flowcharts:** To map out the logical flow of data or the Software Development Lifecycle (SDLC).

10.3. Project Management & Methodology

- **Agile Kanban (Trello):** We are required to create and maintain a **Trello board** to manage our project tasks through different **Sprints**.
- **SDLC (Software Development Lifecycle):** We will follow a structured approach from idea to implementation and delivery.

10.4. Application Architecture (Optional)

- **3-Tier Architecture:** Our proposal for the IT Asset Management system considers the separation of the User Interface, API/Business Logic, and the Database.
- **Design Patterns:** Depending on our specific implementation, we may use patterns like MVC (Model-View-Controller) to organize the application logic.

10.5. Documentation & Version Control

- **GitHub:** Our repository will include a folder for **Documentation** (containing this proposal and diagrams) and a **Scripts** folder (containing the .sql files for schema, seeding, and stored procedures).
- **Trello Board:**
<https://trello.com/invite/b/69fbbd6a98ee9725b4be958f/ATTI8a76f6095fb0d00d3950b522b87fb475FBE9B7F6/my-trello-board>.

11. Team Roles & Responsibilities

- **Lead Database Architect (Nafiseh Khosravi Rad):** Responsible for ERD design, 3NF normalization, and SQL Schema implementation.
- **Security & Compliance Specialist (Kwok Yiu Wong):** Responsible for Authorization Management, Views, and Security Trigger logic.
- **Project Manager / Scrum Master (Danny Gilberto):** Responsible for Trello management, Requirement documentation, and GitHub repository maintenance.